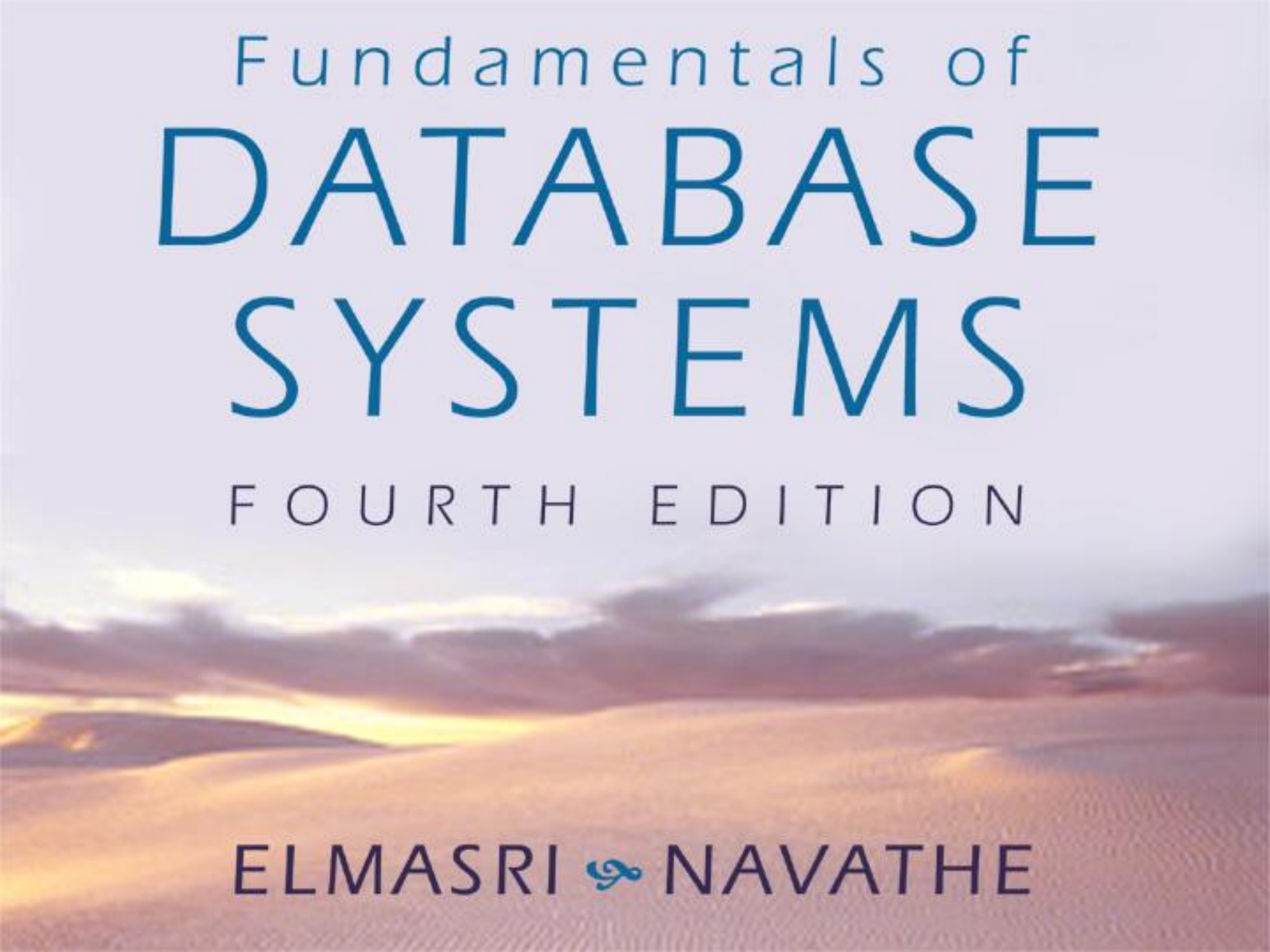


Database Systems

Instructor : Dr. Natash Ali Mian



Switch off mobile phones during lectures, or put them into silent mode



Fundamentals of DATABASE SYSTEMS

FOURTH EDITION

ELMASRI  NAVATHE

Contents

- **Introduction to SQL**
 - DDL Statements
 - DML Statements

Objectives

- Definition of terms
- Interpret history and role of SQL
- Define a database using SQL data definition language
- Write single table queries using SQL
- Establish referential integrity using SQL
- Discuss SQL:1999 and SQL:2003 standards

SQL Overview

- Structured Query Language
- The standard for relational database management systems (RDBMS)
- RDBMS: A database management system that manages data as a collection of tables in which all relationships are represented by common values in related tables

History of SQL

- 1970—E. Codd develops relational database concept
- 1974-1979—System R with Sequel (for Structured English QUERy Language) created at IBM Research Lab, renamed to SQL later
- 1979—Oracle markets first relational DB with SQL
- **1986**—ANSI SQL standard released
- 1989, 1992, 1999, 2003—Major ANSI standard updates
- Current—SQL is supported by most major database vendors. Most DBMS are SQL-99 compliant, with partial SQL-2003 compliant
- Database major players--- Oracle (41%), IBM (31%), Microsoft (13%), MySQL (open source)

Purpose of SQL Standard

- Specify syntax/semantics for data definition and manipulation
- Define data structures and basic operations
- Enable portability
- Specify minimal (level 1) and complete (level 2) standards
- Allow for later growth/enhancement to standard

Benefits of a Standardized Relational Language

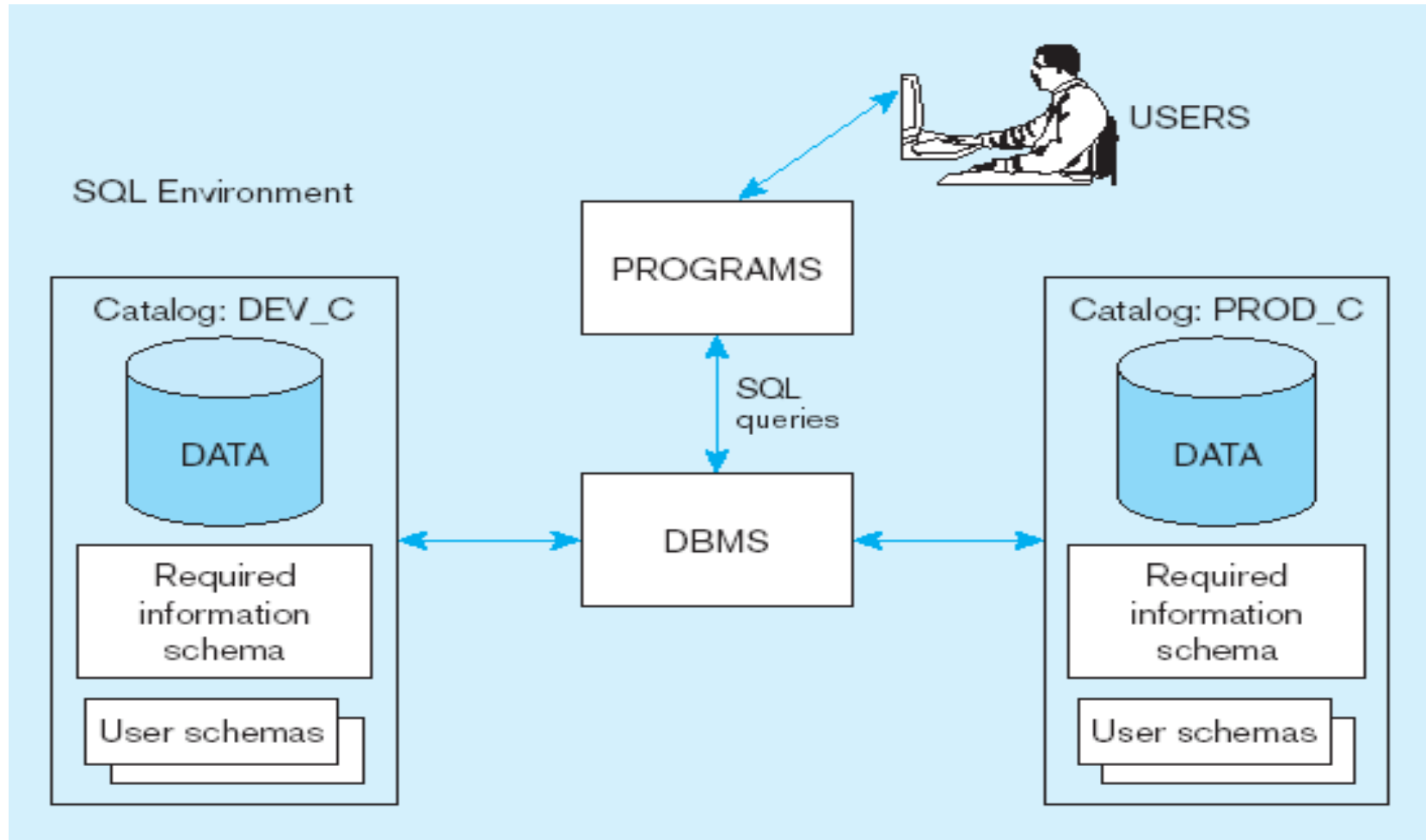
- Reduced training costs
- Productivity
- Application portability
- Application longevity
- Reduced dependence on a single vendor
- Cross-system communication

SQL Environment

- Catalog
 - A set of schemas that constitute the description of a database
- Schema (or Database)
 - The structure that contains descriptions of objects **created by a user** (base tables, views, constraints)
- Data Definition Language (DDL)
 - Commands that define a database, including creating, altering, and dropping tables and establishing constraints
- Data Manipulation Language (DML)
 - Commands that maintain and query a database
- Data Control Language (DCL)
 - Commands that control a database, including administering privileges and committing data

Figure 7-1

A simplified schematic of a typical SQL environment, as described by the SQL-2003 standard



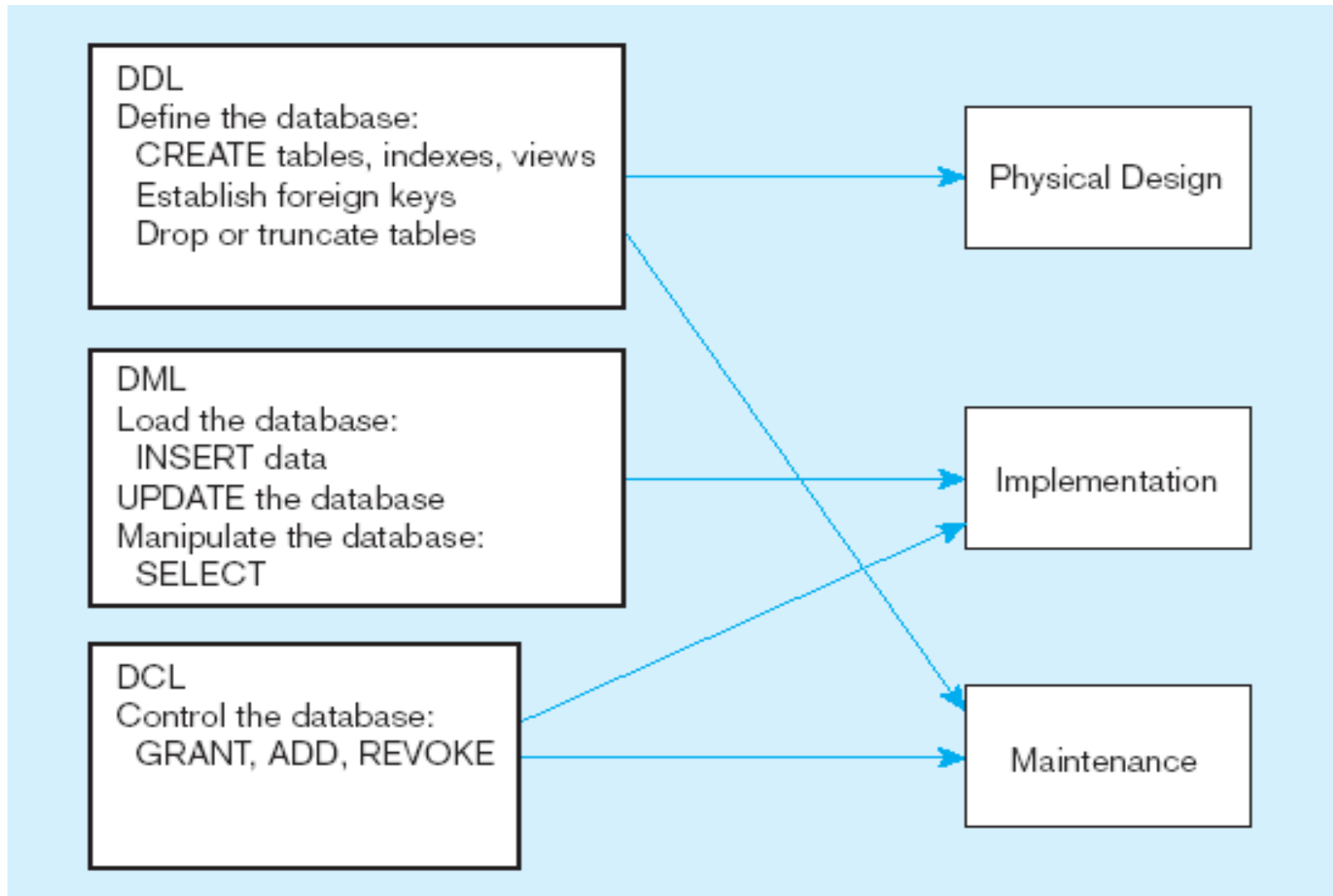
Some SQL Data types

Table 7-2 Sample SQL Data Types

String	CHARACTER (CHAR)	Stores string values containing any characters in a character set. CHAR is defined to be a fixed length.
	CHARACTER VARYING (VARCHAR)	Stores string values containing any characters in a character set, but of definable variable length.
	BINARY LARGE OBJECT (BLOB)	Stores binary string values in hexadecimal format. BLOB is defined to be a variable length.
Number	NUMERIC	Stores exact numbers with a defined precision and scale.
	INTEGER (INT)	Stores exact numbers with a predefined precision and scale of zero.
Temporal	TIMESTAMP	Stores a moment an event occurs, using a definable fraction of a second precision.
Boolean	BOOLEAN	Stores truth values, TRUE, FALSE, or UNKNOWN.

Figure 7-4

DDL, DML, DCL, and the database development process



Common SQL Commands

- **Data Definition Language (DDL):**
 - Create
 - Drop
 - Alter
- **Data Manipulation Language (DML):**
 - Select
 - Update
 - Insert
 - Delete
- **Data Control Language (DCL):**
 - Grant
 - Revoke

Writing SQL Statements

- SQL statements are not case sensitive
(but criteria within quotation marks are for some RDBMS)
- SQL statements can be on one or more lines
- Clauses are usually placed on separate lines
- Keywords cannot be split across lines
- Tabs and spaces are allowed to enhance readability
- Each SQL statement (not line) ends with a semicolon (;)

SQL Database Definition

- Data Definition Language (DDL)
- Major CREATE statements:
 - CREATE SCHEMA (DATABASE)—defines a portion of the database owned by a particular user
 - CREATE TABLE—defines a table and its columns
 - CREATE VIEW—defines a logical table from one or more tables or views

DDL--Table Creation

Figure 7-5 General syntax for CREATE TABLE

```
CREATE TABLE tablename  
( {column definition [table constraint] } , ...  
[ON COMMIT {DELETE | PRESERVE} ROWS] );
```

where *column definition* ::=

```
column_name  
    {domain name | datatype [(size)] }  
    [column_constraint_clause ...]  
    [default value]  
    [collate clause]
```

and *table constraint* ::=

```
[CONSTRAINT constraint_name  
  Constraint_type [constraint_attributes]
```

Steps in table creation:

1. Identify data types for attributes
2. Identify columns that can and cannot be null
3. Identify columns that must be unique (candidate keys)
4. Identify primary key–foreign key mates
5. Determine default values
6. Identify constraints on columns (domain specifications)
7. Create the table and associated indexes

Creating table--constraints

- Domain constraints
 - Constraints about one or more columns
 - NOT NULL, CHECK, DEFAULT
- Entity constraints
 - Constraints about individual rows
 - PRIMARY KEY, UNIQUE
- Referential Integrity constraints
 - FOREIGN KEY

The following slides create tables for this enterprise data model

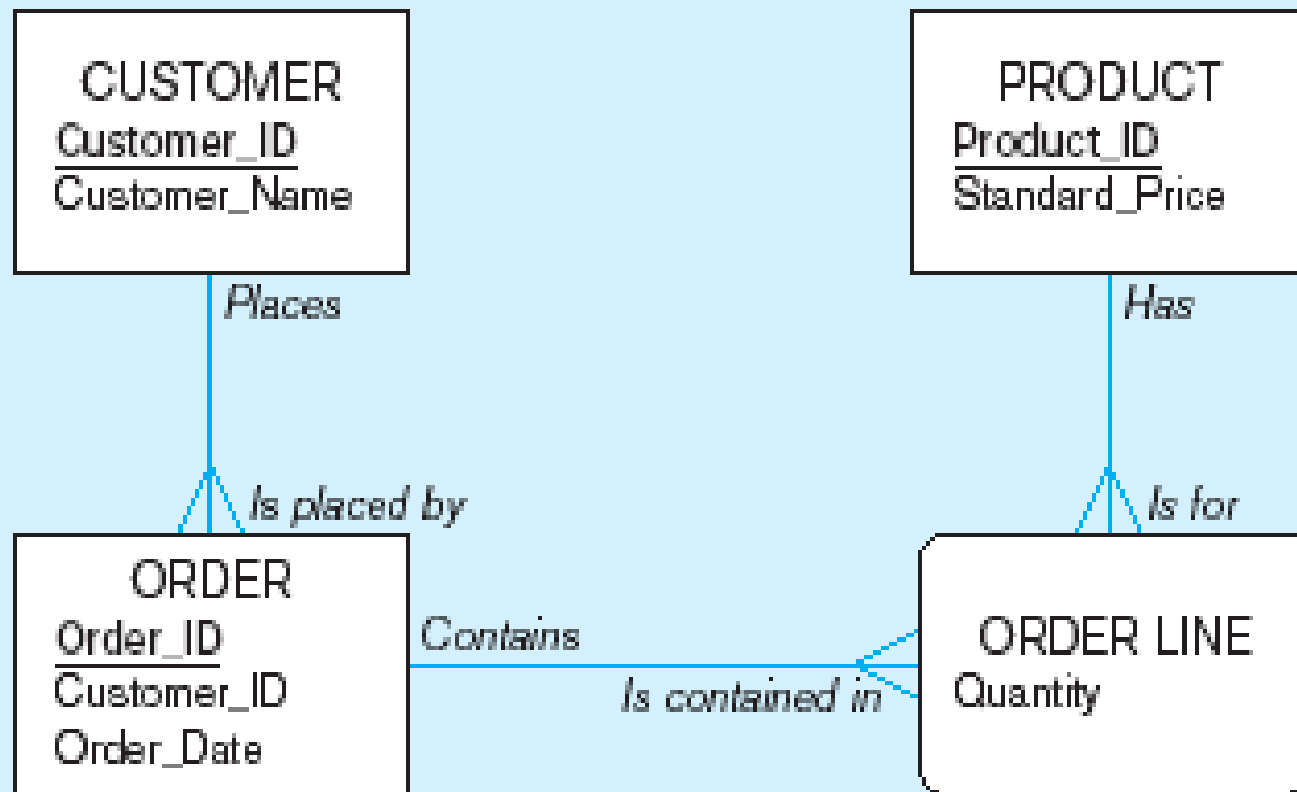


Figure 7-6 SQL database definition commands for Pine Valley Furniture

```
CREATE TABLE CUSTOMER_T
  (CUSTOMER_ID          NUMBER(11, 0) NOT NULL,
   CUSTOMER_NAME        VARCHAR2(25) NOT NULL,
   CUSTOMER_ADDRESS     VARCHAR2(30),
   CITY                 VARCHAR2(20),
   STATE                VARCHAR2(2),
   POSTAL_CODE          VARCHAR2(9),
  CONSTRAINT CUSTOMER_PK PRIMARY KEY (CUSTOMER_ID));
```

Overall table definitions

```
CREATE TABLE ORDER_T
  (ORDER_ID             NUMBER(11, 0) NOT NULL,
   ORDER_DATE           DATE DEFAULT SYSDATE,
   CUSTOMER_ID          NUMBER(11, 0),
  CONSTRAINT ORDER_PK PRIMARY KEY (ORDER_ID),
  CONSTRAINT ORDER_FK FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER_T(CUSTOMER_ID));
```

```
CREATE TABLE PRODUCT_T
  (PRODUCT_ID           INTEGER      NOT NULL,
   PRODUCT_DESCRIPTION   VARCHAR2(50),
   PRODUCT_FINISH        VARCHAR2(20)
                        CHECK (PRODUCT_FINISH IN ('Cherry', 'Natural Ash', 'White Ash',
                                                    'Red Oak', 'Natural Oak', 'Walnut')),
   STANDARD_PRICE        DECIMAL(6,2),
   PRODUCT_LINE_ID       INTEGER,
  CONSTRAINT PRODUCT_PK PRIMARY KEY (PRODUCT_ID));
```

```
CREATE TABLE ORDER_LINE_T
  (ORDER_ID             NUMBER(11,0) NOT NULL,
   PRODUCT_ID           NUMBER(11,0) NOT NULL,
   ORDERED_QUANTITY     NUMBER(11,0),
  CONSTRAINT ORDER_LINE_PK PRIMARY KEY (ORDER_ID, PRODUCT_ID),
  CONSTRAINT ORDER_LINE_FK1 FOREIGN KEY(ORDER_ID) REFERENCES ORDER_T(ORDER_ID),
  CONSTRAINT ORDER_LINE_FK2 FOREIGN KEY (PRODUCT_ID) REFERENCES PRODUCT_T(PRODUCT_ID));
```

Defining attributes and their data types

```
CREATE TABLE PRODUCT_T
```

```
(PRODUCT_ID          INTEGER NOT NULL,
```

```
PRODUCT_DESCRIPTION  VARCHAR2(50),
```

```
PRODUCT_FINISH       VARCHAR2(20)
```

```
        CHECK (PRODUCT_FINISH IN ('Cherry', 'Natural Ash', 'White Ash',  
                                   'Red Oak', 'Natural Oak', 'Walnut')),
```

```
STANDARD_PRICE       DECIMAL(6,2),
```

```
PRODUCT_LINE_ID      INTEGER,
```

```
CONSTRAINT PRODUCT_PK PRIMARY KEY (PRODUCT_ID));
```

```
CREATE TABLE PRODUCT_T
```

Non-nullable specification

```
(PRODUCT_ID          INTEGER NOT NULL,
```

```
PRODUCT_DESCRIPTION  VARCHAR2(50),
```

```
PRODUCT_FINISH       VARCHAR2(20)
```

```
        CHECK (PRODUCT_FINISH IN ('Cherry', 'Natural Ash', 'White Ash',  
                                   'Red Oak', 'Natural Oak', 'Walnut')),
```

```
STANDARD_PRICE       DECIMAL(6,2),
```

```
PRODUCT_LINE_ID      INTEGER,
```

```
CONSTRAINT PRODUCT_PK PRIMARY KEY (PRODUCT_ID));
```

Primary keys
can never have
NULL values

Identifying primary key

```
CREATE TABLE ORDER_LINE_T
```

Non-nullable specifications

```
(ORDER_ID          NUMBER(11,0) NOT NULL,
```

```
PRODUCT_ID        NUMBER(11,0) NOT NULL,
```

```
ORDERED_QUANTITY  NUMBER(11,0),
```

```
CONSTRAINT ORDER_LINE_PK PRIMARY KEY (ORDER_ID, PRODUCT_ID),
```

Primary key

```
CONSTRAINT ORDER_LINE_FK1 FOREIGN KEY (ORDER_ID) REFERENCES ORDER_T (ORDER_ID),
```

```
CONSTRAINT ORDER_LINE_FK2 FOREIGN KEY (PRODUCT_ID) REFERENCES PRODUCT_T (PRODUCT_ID));
```

**Some primary keys are composite—
composed of multiple attributes**

Controlling the values in attributes

```
CREATE TABLE ORDER_T
  (ORDER_ID          NUMBER(11, 0) NOT NULL,
   ORDER_DATE        DATE          DEFAULT SYSDATE,
   CUSTOMER_ID       NUMBER(11, 0),
  CONSTRAINT ORDER_PK PRIMARY KEY (ORDER_ID),
  CONSTRAINT ORDER_FK FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER_T(CUSTOMER_ID));

CREATE TABLE PRODUCT_T
  (PRODUCT_ID        INTEGER      NOT NULL,
   PRODUCT_DESCRIPTION VARCHAR2(50),
   PRODUCT_FINISH     VARCHAR2(20)
   CHECK (PRODUCT_FINISH IN ('Cherry', 'Natural Ash', 'White Ash',
                              'Red Oak', 'Natural Oak', 'Walnut')),
   STANDARD_PRICE     DECIMAL(6,2),
   PRODUCT_LINE_ID    INTEGER,
```

Default value

Domain constraint

Identifying foreign keys and establishing relationships

```
CREATE TABLE CUSTOMER_T
```

```
(CUSTOMER_ID          NUMBER(11, 0) NOT NULL,  
  CUSTOMER_NAME       VARCHAR2(25) NOT NULL,  
  CUSTOMER_ADDRESS    VARCHAR2(30),  
  CITY                VARCHAR2(20),  
  STATE               VARCHAR2(2),  
  POSTAL_CODE         VARCHAR2(9),
```

```
CONSTRAINT CUSTOMER_PK PRIMARY KEY (CUSTOMER_ID));
```

Primary key of
parent table

```
CREATE TABLE ORDER_T
```

```
(ORDER_ID             NUMBER(11, 0) NOT NULL,  
  ORDER_DATE          DATE          DEFAULT SYSDATE,  
  CUSTOMER_ID         NUMBER(11, 0),
```

```
CONSTRAINT ORDER_PK PRIMARY KEY (ORDER_ID),
```

```
CONSTRAINT ORDER_FK FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER_T(CUSTOMER_ID));
```

Foreign key of
dependent table

DDL--Data Integrity Controls

- Referential integrity—constraint that ensures that foreign key values of a table must match primary key values of a related table in 1:M relationships
- Restricting:
 - Deletes of primary records
 - Updates of primary records
 - Inserts of dependent records

DDL--Changing and Removing Tables

- ALTER TABLE statement allows you to change column specifications:
 - ALTER TABLE *table_name* *action*
 - The *action* can be
 - ADD, ALTER or DROP Column,
 - ADD or DROP table constraint
 - Example
 - ALTER TABLE CUSTOMER_T ADD (TYPE VARCHAR(2) default "commercial");
- DROP TABLE statement allows you to remove tables from your schema:
 - DROP TABLE CUSTOMER_T
 - Can be qualified by 'restrict' or 'cascade'

DML--Insert Statement

- Adds data to a table
- Inserting into a table: every attribute is supplied
 - **INSERT INTO CUSTOMER_T VALUES (001, 'Contemporary Casuals', '1355 S. Himes Blvd.', 'Gainesville', 'FL', 32601);**
- Inserting a record that has some null attributes requires entering null explicitly for the empty fields or identifying the fields that actually get data
 - **INSERT INTO PRODUCT_T (PRODUCT_ID, PRODUCT_DESCRIPTION, PRODUCT_FINISH, STANDARD_PRICE, PRODUCT_ON_HAND) VALUES (1, 'End Table', 'Cherry', 175, 8);**
- Inserting from another table
 - **INSERT INTO CA_CUSTOMER_T SELECT * FROM CUSTOMER_T WHERE STATE = 'CA';**

Creating Tables with Identity Columns

New with SQL:2003

```
CREATE TABLE CUSTOMER T
(CUSTOMER_ID INTEGER GENERATED ALWAYS AS IDENTITY
  (START WITH 1
  INCREMENT BY 1
  MINVALUE 1
  MAXVALUE 10000
  NO CYCLE),
CUSTOMER_NAME          VARCHAR (25) NOT NULL,
CUSTOMER_ADDRESS        VARCHAR (30),
CITY                    VARCHAR (20),
STATE                   VARCHAR (2),
POSTAL_CODE             VARCHAR (9),
CONSTRAINT CUSTOMER_PK PRIMARY KEY (CUSTOMER_ID);
```

Inserting into a table does not require explicit customer ID entry or field list

```
INSERT INTO CUSTOMER_T VALUES ( 'Contemporary Casuals',  
'1355 S. Himes Blvd.', 'Gainesville', 'FL', 32601);
```

DML--Delete Statement

- Removes rows from a table
- Delete certain rows
 - **DELETE FROM CUSTOMER_T WHERE STATE = 'HI';**
- Delete all rows
 - **DELETE FROM CUSTOMER_T;**
- Be cautious: always use SELECT clause to display the records first to make sure only desired records are to be deleted!

DML--Update Statement

- Modifies data in existing rows
- **UPDATE PRODUCT_T SET UNIT_PRICE = 775
WHERE PRODUCT_ID = 7;**

Internal Schema Definition

- Control processing/storage efficiency:
 - Choice of indexes
 - File organizations for base tables
 - File organizations for indexes
 - Data clustering
 - Statistics maintenance
- Creating indexes
 - Speed up random/sequential access to base table data
 - Example
 - `CREATE INDEX NAME_IDX ON CUSTOMER_T(CUSTOMER_NAME)`
 - This makes an index for the CUSTOMER_NAME field of the CUSTOMER_T table
 - `DROP INDEX NAME_IDX`

DML--SELECT Statement Syntax

SELECT [DISTINCT] *column_list*
FROM *table_list*
[**WHERE** *conditional expression*]
[**GROUP BY** *column_list*]
[**HAVING** *conditional expression*]
[**ORDER BY** *column_list*];

SELECT Statement

- Used for queries on single or multiple tables
- Clauses of the SELECT statement:
 - **SELECT**
 - List the columns (and expressions) that should be returned from the query
 - **FROM**
 - Indicate the table(s) or view(s) from which data will be obtained
 - **WHERE**
 - Indicate the conditions under which a row will be included in the result
 - **GROUP BY**
 - Indicate categorization of results
 - **HAVING**
 - Indicate the conditions under which a category (group) will be included
 - **ORDER BY**
 - Sorts the result according to specified criteria

SELECT Example

- Find products with standard price less than \$275

```
SELECT PRODUCT_DESCRIPTION, STANDARD_PRICE  
FROM PRODUCT_t  
WHERE STANDARD_PRICE < 275;
```

- When you want to select all columns in the table , use * in the SELECT Clause

```
SELECT *  
FROM PRODUCT_t  
WHERE STANDARD_PRICE < 275;
```

Table 7-3: Comparison Operators in SQL

SELECT Example Using Alias

Alias is an alternative name you give to a column or a table in query

Qualified name: [TABLE NAME].[COLUMN NAME]

```
SELECT CUST.CUSTOMER_NAME AS [NAME],  
       CUST.CUSTOMER_ADDRESS AS [ADDRESS]  
FROM CUSTOMER AS CUST  
WHERE CUSTOMER_NAME= 'Home Furnishings';
```

Aggregate Functions

(Numeric Functions)

COUNT: the number of rows containing the specified column (attribute)

SUM: the sum of all values for a selected column

AVG: the arithmetic mean (average) for the specified column (attribute)

MAX: the largest value in the column

MIN: the smallest value in the column

SELECT Example Using a Function

- Using the COUNT *aggregate function* to find the number of different items that were ordered in an order

```
SELECT COUNT(*) AS #LINEITEMS  
FROM ORDER_LINE_t  
WHERE ORDER_ID = 1004;
```

- Find the average price for all products

```
SELECT AVG (STANDARD_PRICE) AS AVERAGE  
FROM PRODUCT_t;
```

- Find the maximum price for all products

```
SELECT MAX (STANDARD_PRICE) AS MAXIMUM  
FROM PRODUCT_t;
```

- Find the total quantity of products that were ordered in an order

```
SELECT SUM(ORDERED_QUANTITY) AS #ALLITEMS  
FROM ORDER_LINE_t  
WHERE ORDER_ID=1004
```

Using function

- **This is wrong:**

```
SELECT PRODUCT_ID, COUNT(*)  
FROM ORDER_LINE_t  
WHERE ORDER_ID=1004;
```

- While product_id returns row values, count returns a aggregate value. SQL can not return both in a single query

wildcards

- Asterisk (*) in a **SELECT clause** represents all columns or all rows, depending on situation
- Wildcard characters % and _ are often used with keyword LIKE in a **WHERE clause**
- % represents 0, 1 or any number of characters
 - Eg: LIKE '%Desk' matches both 'Computer Desk' and 'Writers Desk'
- Underscore (_) represents exactly one character
 - Eg: LIKE '_-drawer' matches '3-drawer' as well as '5-drawer'

Comparison Operators

- Which orders were placed after 10/24/2006?
 - `SELECT ORDER_ID, ORDER_DATE FROM ORDER_t WHERE ORDER_DATE > '24-OCT-2006'`
- What furniture aren't made of cherry?
 - `SELECT PRODUCT_DESCRIPTION, PRODUCT_FINISH FROM PRODUCT_t WHERE PRODUCT_FINISH != 'Cherry'`

Table 7-3 Comparison Operators in SQL

Operator	Meaning
=	Equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
<>	Not equal to
!=	Not equal to

Boolean operators

- Used in **WHERE clause** to join two conditions
- And: returns results only when all conditions are true
- OR: returns results when any conditions are true
- NOT: negates a expression
- Order of the operators: NOT first, then AND, then OR
- Use parenthesis to change default order of operators

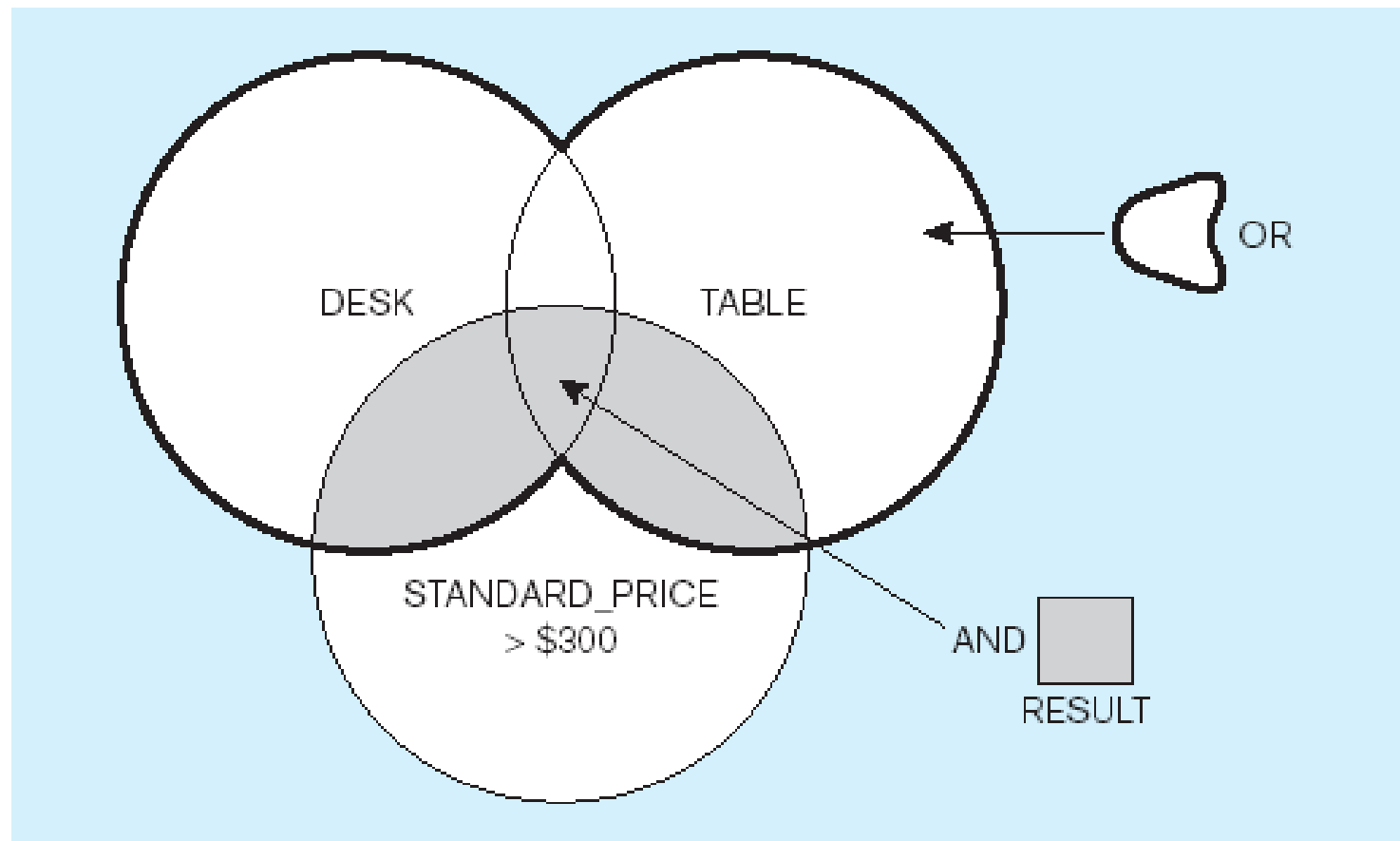
SELECT Example–Boolean Operators

- **AND**, **OR**, and **NOT** Operators for customizing conditions in WHERE clause

```
SELECT PRODUCT_DESCRIPTION, PRODUCT_FINISH,  
       STANDARD_PRICE  
FROM PRODUCT_t  
WHERE (PRODUCT_DESCRIPTION LIKE '%Desk'  
OR PRODUCT_DESCRIPTION LIKE '%Table')  
AND UNIT_PRICE > 300;
```

Note: the LIKE operator allows you to compare strings using wildcards. For example, the % wildcard in '%Desk' indicates that all strings that have any number of characters preceding the word "Desk" will be allowed

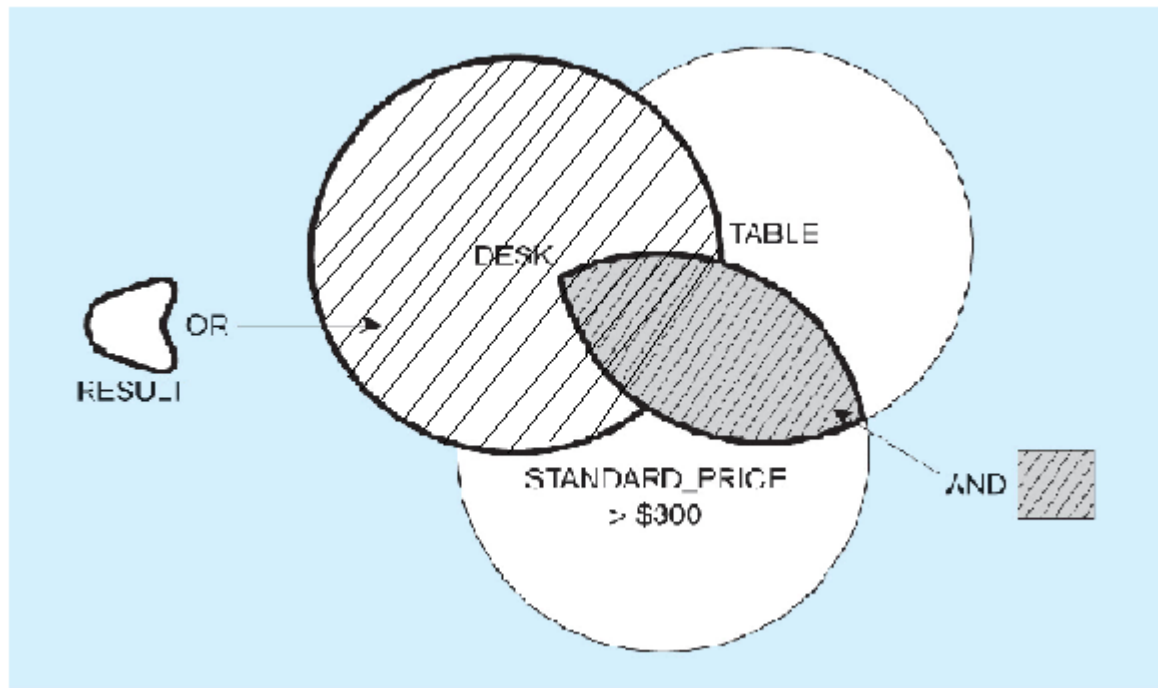
Venn Diagram from Previous Query



SELECT Example – Boolean Operators

- Note this query is different from the previous example
SELECT PRODUCT_DESCRIPTION, PRODUCT_FINISH,
STANDARD_PRICE
FROM PRODUCT_t
WHERE PRODUCT_DESCRIPTION **LIKE** '%Desk'
OR PRODUCT_DESCRIPTION **LIKE** '%Table'
AND UNIT_PRICE > 300;

Venn Diagram from Previous Query



Using ranges for qualification

- Which products have a standard price between 200 and 300?

```
SELECT PRODUCT_DESCRIPTION, STANDARD_PRICE FROM  
    PRODUCT_t  
    WHERE STANDARD_PRICE>199 AND STANDARD_PRICE<301;
```

```
SELECT PRODUCT_DESCRIPTION, STANDARD_PRICE FROM  
    PRODUCT_t  
    WHERE STANDARD_PRICE BETWEEN 200 AND 300;
```

- BETWEEN is inclusive (include 200 and 300)

Distinct values

- What are the distinct order numbers included in the ORDER_LINE table?

```
SELECT DISTINCT ORDER_ID  
FROM ORDER_LINE_t;
```

- What are the unique combinations of order number and order quantity included in the ORDER_LINE table?

```
SELECT DISTINCT ORDER_ID, ORDERED_QUANTITY  
FROM ORDER_LINE_t;
```

Using IN and NOT IN with list

- List all customers live in warmer states

```
SELECT CUSTOMER_NAME, CITY, STATE  
FROM CUSTOMER_t  
WHERE STATE IN ('FL', 'TX', 'CA', 'HI');
```

SELECT Example –

Sorting Results with the ORDER BY Clause

- Sort the results first by STATE, and within a state by CUSTOMER_NAME

```
SELECT CUSTOMER_NAME, CITY, STATE  
FROM CUSTOMER_t  
WHERE STATE IN ('FL', 'TX', 'CA', 'HI')  
ORDER BY STATE, CUSTOMER_NAME;
```

Note: the IN operator in this example allows you to include rows whose STATE value is either FL, TX, CA, or HI. It is more efficient than separate OR conditions

SELECT Example–

Categorizing Results Using the GROUP BY Clause

- For use with aggregate functions
 - Divide table into subsets (by groups), then aggregate function values are calculated for each group
 - *Scalar aggregate*: single value returned from SQL query with aggregate function
 - *Vector aggregate*: multiple values returned from SQL query with aggregate function (via GROUP BY)
- Eg: count the number of customers in each state

```
SELECT STATE, COUNT(STATE) AS NO_CUSTOMERS  
FROM CUSTOMER_t  
GROUP BY STATE;
```

SELECT Example– Qualifying Results by Categories Using the **HAVING Clause**

- For use with GROUP BY
- Similar to a WHERE clause, except that the restriction is applied on groups instead of on rows
- Eg: Find the states with more than one customers

```
SELECT STATE, COUNT(STATE) AS NO_CUSTOMERS  
FROM CUSTOMER_t  
GROUP BY STATE  
HAVING COUNT(STATE) > 1;
```

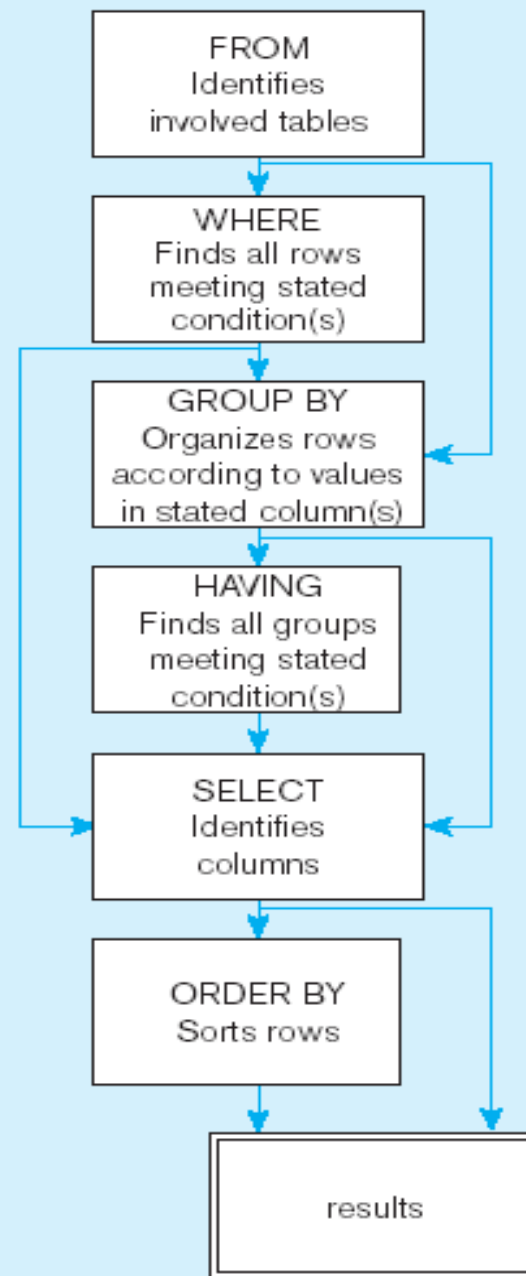
Like a WHERE clause, but it operates on groups (categories), not on individual rows. Here, only those groups with total numbers greater than 1 will be included in final result

SELECT Example – Qualifying Results by Categories Using the **HAVING Clause**

- Example: list the product finish and average standard price for selected finishes where the average standard price is less than 750

```
SELECT Product_Finish, AVG(Standard_Price) AS AVG_PRICE
FROM   PRODUCT
WHERE Product_Finish IN ('Cherry', 'Natural Ash', 'Natural Maple', 'White
Ash')
GROUP BY Product_Finish
HAVING (AVG(Standard_Price) < 750)
ORDER BY Product_Finish
```

Figure 7-10
SQL statement
processing
order (adapted
from van der
Lans, p.100)



Using and Defining Views

- Views simplify query command, provide users controlled access to tables
- Base Table—table containing the raw data
- Dynamic View
 - A “virtual table” created dynamically upon request by a user
 - No data actually stored; instead data from base table made available to user
 - Based on SQL SELECT statement on base tables or other views
 - You can think of views as stored queries
- Materialized View
 - Copy or replication of data
 - Data actually stored
 - Must be refreshed periodically to match the corresponding base tables

Sample CREATE VIEW

```
CREATE VIEW EXPENSIVE_STUFF_V AS  
SELECT PRODUCT_ID, PRODUCT_NAME, UNIT_PRICE  
FROM PRODUCT_T  
WHERE UNIT_PRICE > 300  
WITH CHECK OPTION;
```

- View has a name
- View is based on a SELECT statement
- CHECK OPTION—applied to updatable view

Sample CREATE VIEW

- Update of view will result in a update of the base table from which the view is derived
- Check options
 - Cause the UPDATE or INSERT statement to be rejected if the statements would cause updated or inserted rows to be removed from the view.
 - Eg: the following statement will be rejected
UPDATE EXPENSIVE_STUFF_V
SET STANDARD_PRICE=295
WHERE PRODUCT_ID=5;

Advantages of Views

- Simplify query commands
- Assist with data security (but don't rely on views for security, there are more important security measures)
- Enhance programming productivity
- Contain most current base table data
- Use little storage space
- Provide customized view for user
- Establish physical data independence

Disadvantages of Views

- Use processing time each time view is referenced
- May or may not be directly updateable

Thanks